February 11, 2026

Department of Computer Science

Reinforcement Learning (Dr. Aya Kayal)
Coursework based on the design of Dr. Manon Flageat
Teaching Assistants: Haidar Jomaa, Wafaa El Bokaily

Assessed Coursework 1

The Coursework should be submitted on Moodle by February 25th, 7 pm, and consist in:
- A **PDF** of your written report, imperatively named ***coursework1_report.pdf***.
- A **Python file** of the final code used to generate your results, based on the provided code, imperatively named ***coursework1.py***. You can download it in this format from Colab.
Unfortunately, emailed or printed submissions cannot be accepted.

The report document should be preferably typeset, ideally in Latex. Your answers should be yours, i.e., written by you, in your own words, showing your own understanding. Written answers should be clear, complete and concise. Figures should be clearly readable, labelled and visible. Poorly produced answers, irrelevant text not addressing the point and unclear text may loose points. As a rule of thumb if the work is not presented in such a way that the markers can easily verify your understanding, it will be difficult to get marks for it. Note, that the coursework will be graded on the written report, submitted code is treated as ancillary documentation of your authorship, not as a substitute for a written answer.
Your coursework submission should not be longer than 10 single sided A4 pages with at least 2 centimetre margins all around and 12pt font.
You should produce your own code to compute your specific answers. If you have questions about the coursework please ask TAs, but they cannot provide you with answers that directly solve the coursework.

Marks are shown next to each question. Note, that the marks are only indicative.

Exercise 1: Understanding of MDPs (16 points)

In this question, all students will use the same trace generated from CID = 01897694.

Consider the following observed traces for a Markov Decision Process with four states $\mathbb{S} = \{s_0, s_1, s_2, s_3\}$, discount factor $\gamma = 1$, and immediate rewards specified after every state.

There is only one possible action ("no choice"), which is taken at every step. Therefore, the state transitions depend only on the transition dynamics, and actions are omitted from the trace.

The traces are composed of a state $s(t)$ at time step t and an immediate reward $r(t)$ collected upon departing from state $s(t)$. The reward $r(t)$ corresponds to leaving state $s(t)$.

- Remove the leading zero from CID = 01897694 and denote the digits from left to right as $CID(1), CID(2), \ldots$

- For $t = 1, 2, \ldots,$ define

$$s(t) = (\text{CID}(t) + 2) \mod 4 \tag{1}$$

and

$$r(t) = \text{CID}(t) \mod 4, \tag{2}$$

where *mod* denotes the modulo operation.[1]

The trace ends at the last observed state; any reward after the last state is omitted. We do not assume whether the last state is terminal or transient; the trace simply represents the observed data.

a. Write the trace (sequence of states and rewards) generated by CID = 01897694. (1 pt)

b. Draw a minimal MDP graph consistent with the observed data (do not add transitions that are not supported by the data). Please include any self-transitions. Briefly explain your rationale for the graph you drew. (4 pts)

c. Assume that the transition matrix and reward function of this MDP are unknown, as would be the case for a naive reinforcement learning agent. (11 pts)

   1. Provide an expression for the transition matrix that can be inferred from the data or from the graph you constructed.
   2. What can you infer about the structure of the reward function and the reward matrix? Briefly explain.
   3. What can you compute about the value of the first state of the trace $s(t = 1)$? Name any method you use and justify your choice.

Exercise 2: Understanding of the Maze Environment (84 points)

The focus of this part of the Coursework is the resolution of a Maze environment, illustrated in Figure 1 and model as a Markov Decision Process (MDP). In this illustration, the black squares symbolise obstacles, and the dark-grey squares absorbing states, that correspond to specific rewards. Absorbing states are terminal states, there is no transition from an absorbing state to any other state.

This environment shares a lot of similarities with the Grid World environment studied in the practice assignment. However, unlike the practice assignment, this part of the Coursework aims to help you implement RL techniques that do not require full knowledge of the dynamic of the environment, namely Monte-Carlo and Temporal-Difference learning. In the following, we will thus assume we don't have access to the transition matrix T and reward function R of the environment, and try to find the optimal policy π by sampling episodes in it.

Consider an agent moving inside this maze, trying to get the best possible reward. To do so, it can choose at any time-step between four actions:

- $a_0 =$ *going north* of its current state
- $a_1 =$ *going east* of its current state
- $a_2 =$ *going south* of its current state

[1] `https://en.wikipedia.org/wiki/Modulo_operation`

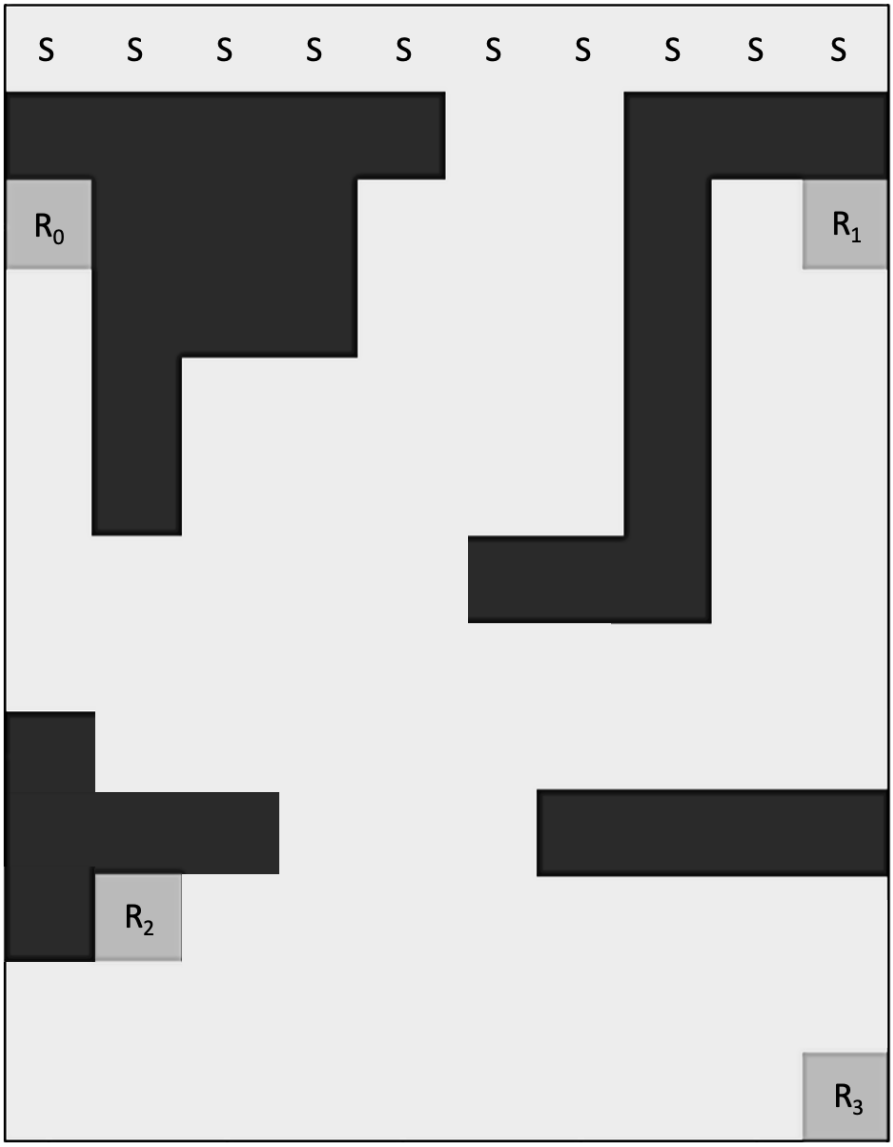

Figure 1: Illustration of the Maze environment, the focus of this part of the Coursework. The absorbing states are indicated as "R_i", they correspond to absorbing rewards. The "S" indicate the potential start states.

- $a_3 = \textit{going west}$ of its current state

The chosen action has a probability p to succeed and lead to the expected direction. If it fails, it has equal probability to lead to any other direction. For example, if the agent chooses the action $a_0 = \textit{going north}$, it is going to succeed and go north with probability p, and it has probability $\frac{1-p}{3}$ to go east, probability $\frac{1-p}{3}$ to go south and probability $\frac{1-p}{3}$ to go west. p is given as a hyperparameter, defining the environment.

If the outcome of an action leads the agent to a wall or an obstacle, its effect is to keep the agent where it is. For example, if the agent chooses the action $a_1 = \textit{going east}$ and there is a wall east but no wall in the other directions, it is going to stay in place in his current state with probability p and to go north, south or west with the same probability $\frac{1-p}{3}$.

Any action performed by the agent in the environment gives it a reward of -1.

The episode ends when the agent reach one of the absorbing state $(R_i)_{0 \leq i \leq 3}$ or if it performs more than 500 steps in the environment.

Finally, the starting state is chosen randomly at the beginning of each episode from the possible start states with equal probability $1/N_{start-states}$. The possible start states are indicated on the Maze illustration as "S".

For this exercise, use the College ID 1610093. Let y and z be the last two digits of this ID: $y = 9$, $z = 3$

- The probability p is set as $p = 0.8 + 0.02 \times (9 - y)$.
- The discount factor γ is set as $\gamma = 0.8 + 0.02 \times y$.
- The reward state is R_i with $i = z \; mod \; 4$
- The reward state R_i gives a reward $r = 500$ and all the other absorbing states gives a reward $r = -50$. As stated before, any action performed by the agent in the environment also gives it a reward of -1.

Jupyter Notebook

This part of the Coursework is based on the provided Jupyter Notebook, also available on Colab: `https://colab.research.google.com/drive/1hb27DGoBZliCf85ShWH-zb8hPb_Uhy3O#scrollTo=3ucYXx5NqSt`
It defines most of the Maze structure. In the following questions, you will be asked to progressively complete the functions of the Maze class indicated with the tag "[Action required]".

The Notebook has the following structure:

1. `get_CID()` and `get_login()` functions definition. Everyone use the CID 1610093.
2. `GraphicsMaze` class definition: allow all the graphical visualisation of the Maze. You **DO NOT NEED** to read or understand this class, you can simply call its methods when needed.
3. `Maze` class definition: this class is the main Maze class, you will be asked to complete it in the following questions. All attributes and methods of this class starting with `_` such as `env._T` or `env._build_maze()` are private and should NOT be accessed outside the `Maze` class. `Get` functions such as `get_T()` are defined to access some of them from outside the class.
4. `DP_agent` class definition: this class defined a Dynamic Programming agent to solve the Maze, you will be asked to complete it in the following questions.
5. `MC_agent` class definition: this class defined a Monte-Carlo Learning agent to solve the Maze, you will be asked to complete it in the following questions.
6. `TD_agent` class definition: this class defined a Temporal-Difference Learning agent to solve the Maze, you will be asked to complete it in the following questions.

Question 0: Defining the environment – 2 pts

We are first going to define the Maze environment used in the following.

[Report] – 2 pts

Fill in the following part of the Jupyter Notebook:

- The `__init__()` method of the `Maze` class, to set your reward state, absorbing reward, p, and γ in the environment.

State all these values in the report.

Question 1: Dynamic programming – 14 pts

For comparison purposes, we are first going to assume we have access to the transition T and reward function R of this environment and consider a Dynamic Programming agent to solve this problem.

Complete the `solve()` method of the `DP_agent` class. You can choose any Dynamic Programming method. Note that *for this agent only*, you are allowed to access the transition matrix through `env.get_T()`, the reward matrix through `env.get_R()` and the list of absorbing states through `env.get_absorbing()` of the `Maze` class. You might also need to use `env.get_action_size()`, `env.get_state_size()` and `env.get_gamma()`.

You can define any additional methods inside the `DP_agent` class, but only the `solve()` method will be called. **DO NOT** add any inputs or outputs to `solve()`.

[Report] – 14 pts

1. State which method you choose to solve the problem and justify your choice. Give and justify any parameters that you set, design that you chose or assumptions you made. **– 4 pts**
2. Report the graphical representation of your optimal policy and optimal value function. **– 6 pts**
3. Discuss how the value of your γ and p have influenced the optimal value function and optimal policy in your personal Maze. In particular, you may investigate the effect of having a value of $p < 0.25$, $p = 0.25$ or $p > 0.25$, and similarly $\gamma < 0.5$ or $\gamma > 0.5$. **– 4 pts**

Question 2: Monte-Carlo Reinforcement Learning – 28 pts

We are now assuming that we do not have access to the transition T and reward function R of this environment. We want to solve this problem using a Monte-Carlo learning agent.

Complete the `solve()` method of the `MC_agent` class. Note that for this agent you are only allowed to use the `env.reset()` and `env.step()` methods of the `Maze` class, as well as `env.get_action_size()`, `env.get_state_size()` and `env.get_gamma()`. **DO NOT** use the transition matrix `env.get_T()`, the reward matrix `env.get_R()` and the list of absorbing states `env.get_absorbing()`.

You can define any additional methods inside the `MC_agent` class, but only the `solve()` method will be called. **DO NOT** add any inputs or outputs to `solve()`.

[Report] – 28 pts

1. State which method you choose to solve the problem and justify your choice. Give and justify any parameters that you set, design that you chose or assumptions you made. **– 8 pts**
2. Report the graphical representation of your optimal policy and optimal value function. **– 6 pts**
3. Multiple MC-learning runs will lead to slightly different results due to variability in the learning, and you would need to replicate the learning process multiple times to get an idea of the true performance of your agent. Identify the sources of this variability, and state what a "sufficient" number of replications would be, explaining how you would choose it. **– 4 pts**
4. Plot the learning curve of your agent: the total non-discounted sum of reward against the number of episodes. The figure should picture the mean and standard deviation across your replications on the same graph (using the replication number you stated in the previous question). **– 4 pts**

5. How do the exploration parameter ε and the discount factor γ affect your learning? Explore a range of values, including extreme ones, and illustrate their impact on learning curves and final policies with relevant plots. Briefly explain your findings and relate them to the theory you learned. We encourage you to use relevant additional plots to illustrate your findings in this question. **– 6 pts**

Question 3: Temporal Difference Reinforcement Learning – 24 pts

We are now going to assume we do not have access to the transition T and reward function R of this environment. We want to solve this problem using a Temporal-Difference learning agent.

Complete the `solve()` method of the `TD_agent` class. Note that for this agent you are only allowed to use the `env.reset()` and `env.step()` methods of the `Maze` class, as well as `env.get_action_size()`, `env.get_state_size()` and `env.get_gamma()`. **DO NOT** use the transition matrix `env.get_T()`, the reward matrix `env.get_R()` and the list of absorbing states `env.get_absorbing()`.

You can define any additional methods inside the `TD_agent` class, but only the `solve()` method will be called. **DO NOT** add any inputs or outputs to `solve()`.

[Report] – 24 pts

1. State which method you choose to solve the problem and justify your choice. Give and justify any parameters that you set, design that you chose or assumptions you made. **– 8 pts**
2. Report the graphical representation of your optimal policy and optimal value function. **– 6 pts**
3. Plot the learning curve of your agent: the total non-discounted sum of reward against the number of episodes. The figure should picture the mean and standard deviation across your replications on the same graph (using the replication number you stated in Question 2: Monte-Carlo Reinforcement Learning). **– 4 pts**
4. How does varying the exploration parameter ϵ and the learning rate α of your algorithm impact your learning curves? Briefly explain what you find and relate it where possible to the theory you learned. We encouraged you to use relevant additional plots to illustrate your findings in this question. **– 6 pts**

Question 4: Comparison of learners – 16 pts

We now compare the Monte-Carlo (MC) and Temporal-Difference (TD) approaches, using the Dynamic Programming results as a Baseline.

[Report] – 16 pts

1. We define the value function estimation error as the mean square error between the optimal value function vector computed through Dynamic Programming and the optimal value function vector computed through one of the learners. Compute this estimation error for both TD and MC learners, and plot them on the same graph against the number of episodes (you can use any built-in function such as `mean_squared_error` from `sklearn.metrics`). The plot should also picture the standard deviation for both of these graphs (using the same number of replications as in the previous questions). **– 4 pts**
2. Provide an analysis of the comparison you plotted in the previous question. Relate your reasoning to the theory of the two approaches. **– 4 pts**

3. We use the same definition for the function estimation error. For one given run, plot as a scatter plot the estimation error for an episode against the total non-discounted sum of reward for that episode for both TD and MC learners. **– 4 pts**

4. Explain what this new plot characterises. Based on it, explain how important it is to have a good value function estimate to obtain a good reward. What conclusions can you draw? Relate your reasoning to the theory of the two approaches and mention briefly how the hyperparameters influence this graph. **– 4 pts**